Action – Inviare e ricevere messaggi tra nodi

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti
Tutor: **Rita Molle**
email: rita.molle@unicampus.it

CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

CREO LAB

UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA

Outline

- Recap
- Esercitazione
- Scrivere un'interfaccia personalizzata
- Scrivere un action server → python
- Scrivere un action client → python

Recap topic e service

Topic

- Usati dai nodi per inviare e ricevere messaggi
- I publisher pubblicano i dati su un topic
- I subscriber sottoscrivono al topic per ricevere dati

Service

- Usati per interazioni tra nodi client/server
- Il client invia una richiesta al server
- Il server processa la richiesta e restituisce una risposta al client



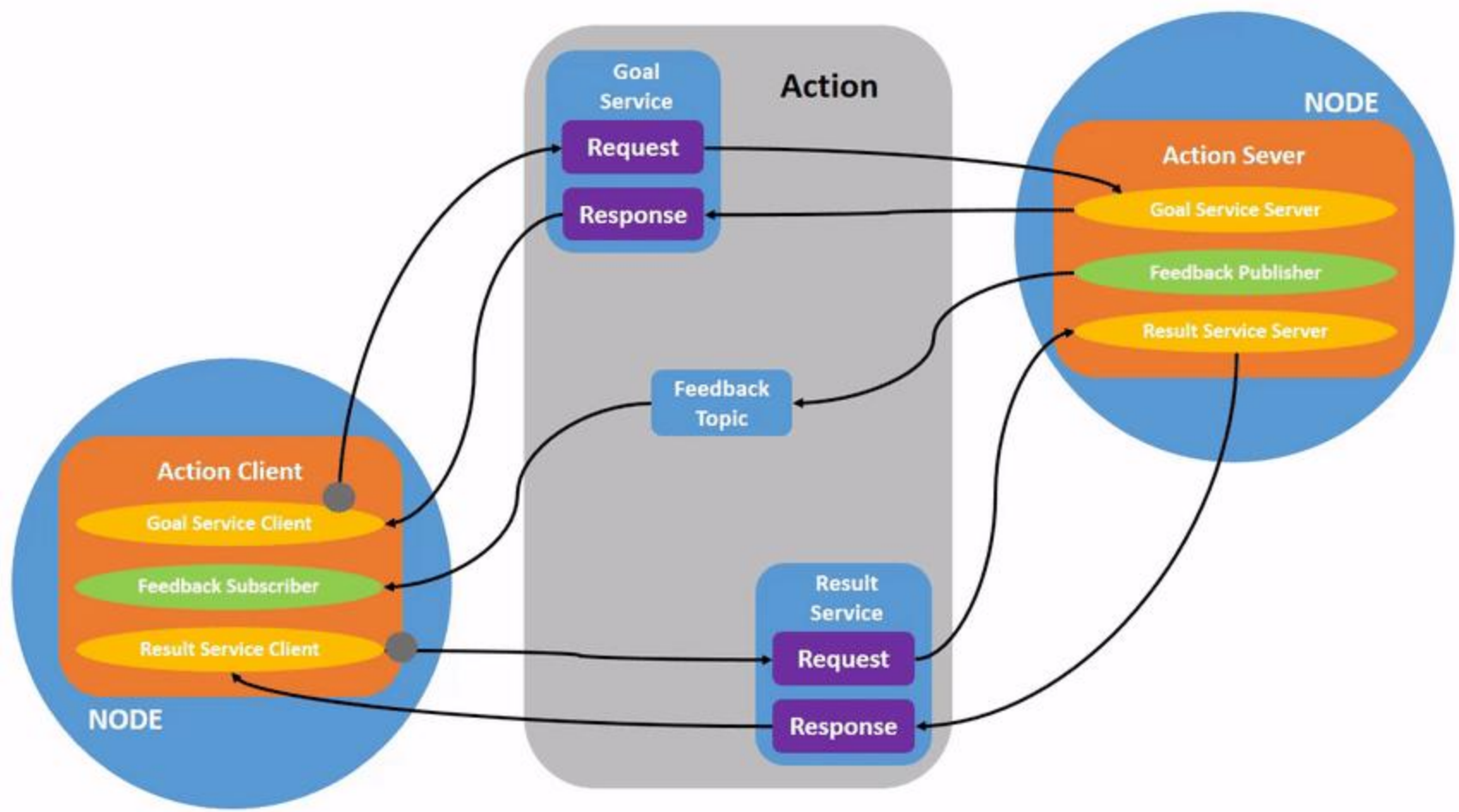
Se si vuole muovere una parte fisica di un robot → no service, perchè:

- Richiede tempo di esecuzione
- Il client non vede il server → no feedback istantaneo
- Il client non può interrompere un'esecuzione
- Con richiesta multiple, come si comporta il server?

Uso di **action** { meccanismo di feedback; meccanismo di "cancel"

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Recap


Goal
Service
Request
Response
Action
NODE
Action Sever
Goal Service Server
Feedback Publisher
Result Service Server
Feedback
Topic
Action Client
Goal Service Client
Feedback Subscriber
Result Service Client
NODE
Result
Service
Request
Response

Recap

- Si usano i topic per inviare data streams tra nodi → no risposta
- Si usano i service per comunicazioni server/client (esecuzioni semplici)
- Si usano le actions per comunicazioni server/client (esecuzioni che richiedono tempo, feedback, cancel)

- Un'azione è definita da name e interface (goal, result, feedback)
- Client e server hanno la stessa interface
- Si possono inviare più goal da uno o più action client
- Action client non conoscono il nodo con il server

- Inviare un goal dal client al server
- Accettare o no il goal ed eseguirlo sul server
- Finito il goal, restituire un risultato dal server al client
- Inviare feedback di esecuzione dal server al client
- Permettere al client di inviare una richiesta di cancel

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Recap

- Creare interface
- Creare action server
  - Goal callback: il server riceve un goal e decide se accettarlo
  - Execute callback: esecuzione del goal; pubblicare feedback (opzionale)
  - Cancel callback (opzionale): si riceve una richiesta di cancel
- Creare action client
  - Scrivere un metodo per inviare un goal
    - Goal response callback: per sapere se il goal è accettato o meno dal server
    - Goal result callback: ottieni il risultato finale
    - Feedback callback (opzionale): per ricevere i feedback dal server

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Recap topic, service, action

TIPO	DURATA	RISPOSTA	FEEDBACK	USO
Topic	Continua	✗	✗	Sensori
Service	Breve	✓	✗	Trigger
Action	Lunga	✓	✓	Movimenti

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Esercitazione:

- Realizzare un sistema ROS2 basato su Action che permetta di controllare il movimento oscillatorio della testa di un robot
  - Creare interfaccia personalizzata
  - Creare action server /move_head → riceve goal, muove la testa, usa FollowJointTrajectory per controllare i giunti; invia feedback continuo; restituisce success=True
  - Creare action client → si connette a /move_head, invia goal, riceve feedback in tempo reale e risultato finale

Terminale: 

```
ros2 action list –t
ros2 action info /head_controller/follow_joint_trajectory
ros2 interface show control_msgs/action/FollowJointTrajectory
```


E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Creare una action interface

Action server: muove la testa tra due limiti con un passo → restituisce il risultato al client quando ha finito
Action client: invia al server i limiti del movimento e il passo

Che tipo di interface??? Si ragiona su goal, result, feedback
→ goal: posizione minima, posizione massima, passo
→ result: booleano
→ feedback: posizione corrente della testa (float)

Per le actions non esistono molte interface → crearne una nel my_robot_interfaces package

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Creare una action interface

Creare la folder action nel pkg my_robot_interfaces

Terminal 1:

```
cd ~/progetto_n_ws/src
ros2 pkg create my_robot_interfaces --build-type ament_cmake
cd my_robot_interfaces
mkdir action
```

Creare un nuovo file per l'action. Nel filename:

- Usare UpperCamelCase
- Non scrivere Action o Interface nel nome
- Usare .action per l'estensione
- Usare un verbo nel nome dell'interface

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Creare una action interface

Terminal 1: cd action
touch MoveHead.action

Aprire il file e scrivere l'interface → aggiungere --- per separare goal/result; result/feedback

```
#Goal
float64 min
float64 max
float64 step
---
#Result
bool success
---
#Feedback
float64 current_position
```


E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Creare una action interface

Aprire il file CMakeLists.txt

Dopo find_package (ament_camake REQUIRED) e prima di ament_package()

modificare:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/MoveHead.action"
)
```

Aprire il file package.xml e aggiungere

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

Salvare

Terminal 1: 

```
colcon build --packages-select my_robot_interfaces
source ~/.bashrc
```


E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ